



Градиентный
бустинг и со.



Ансамбли повторение

Идея бустинга. AdaBoost

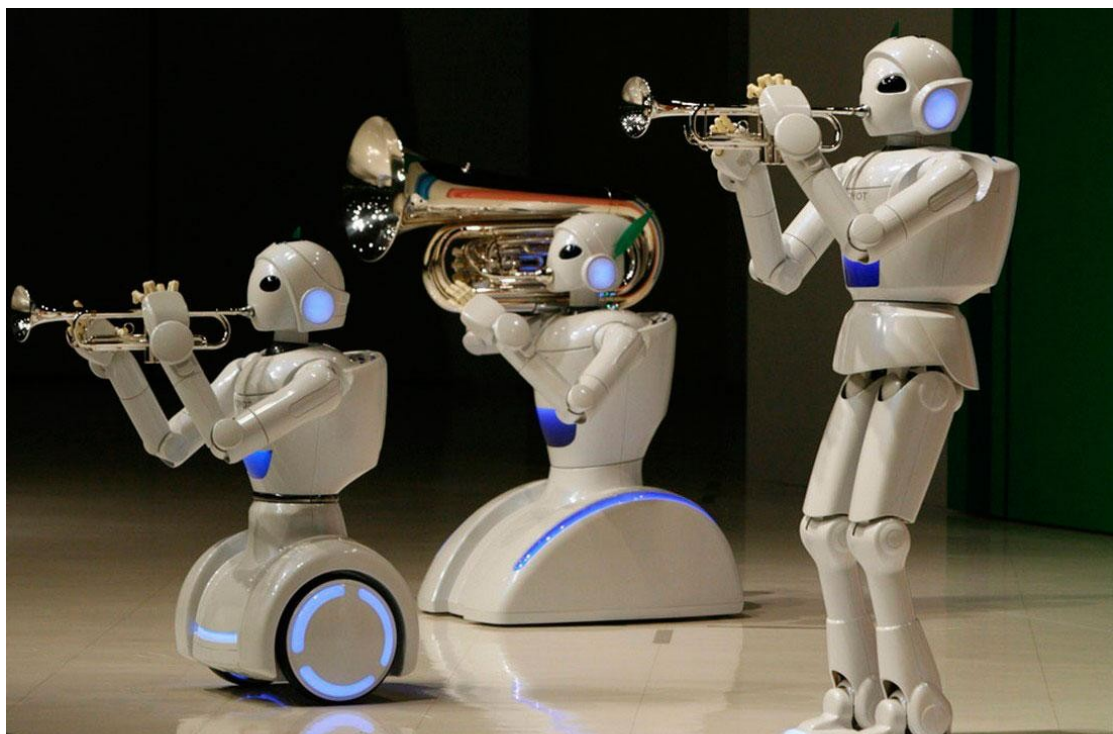
Градиентный бустинг

Семья бустингов

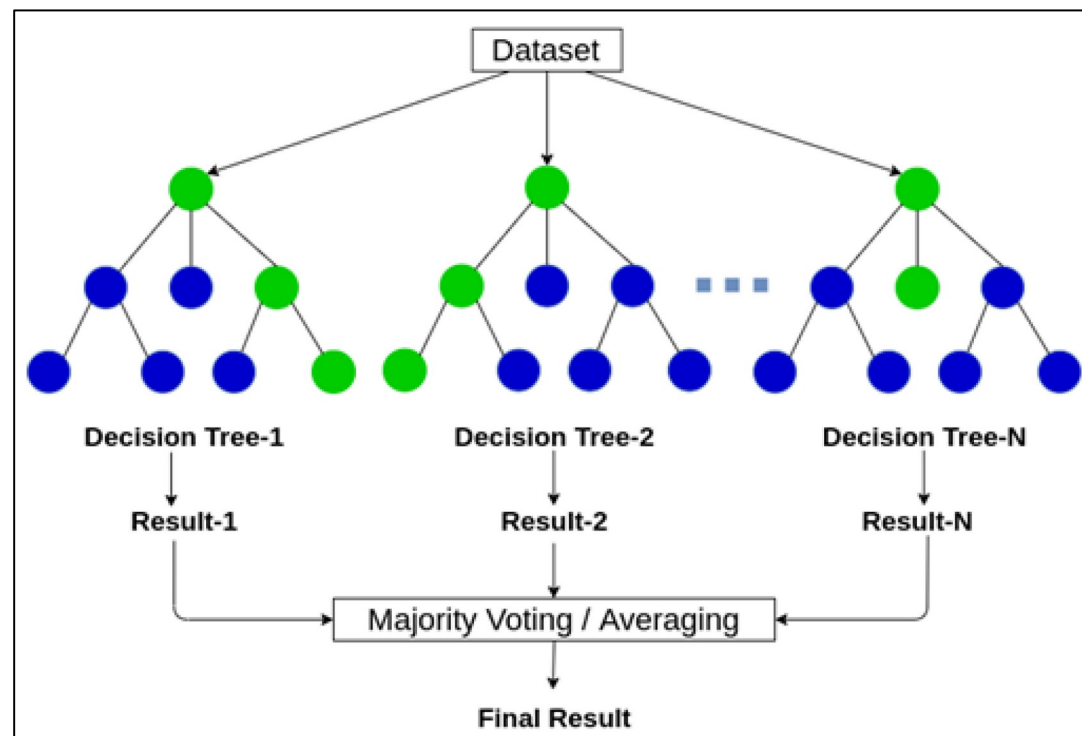
Стеккинг

Ансамбли в машинном обучении

Ожидание

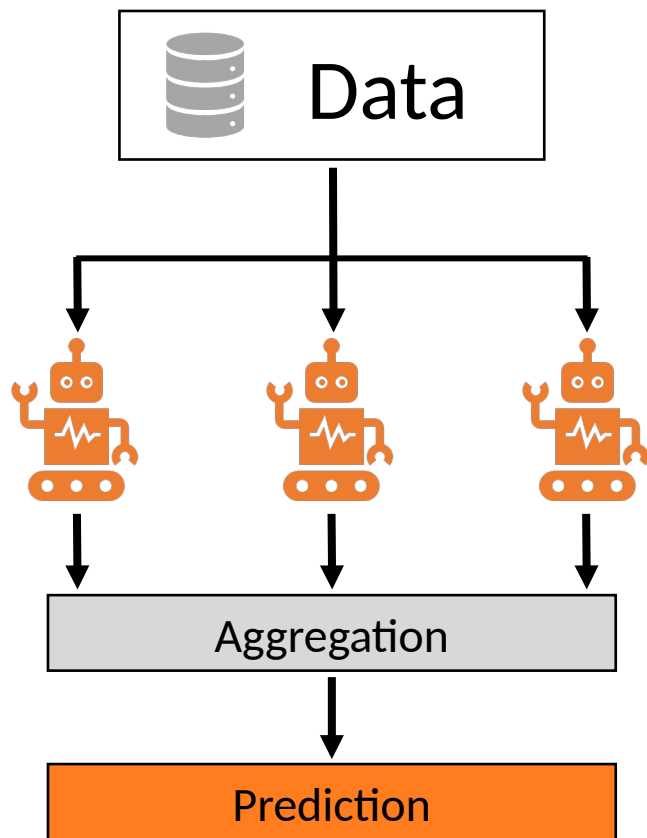


Реальность

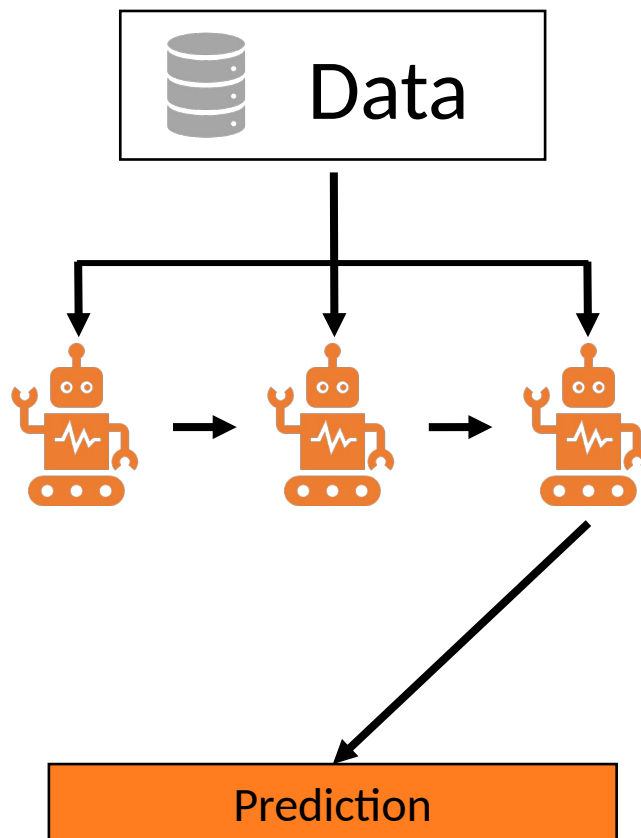


Ансамбли в машинном обучении

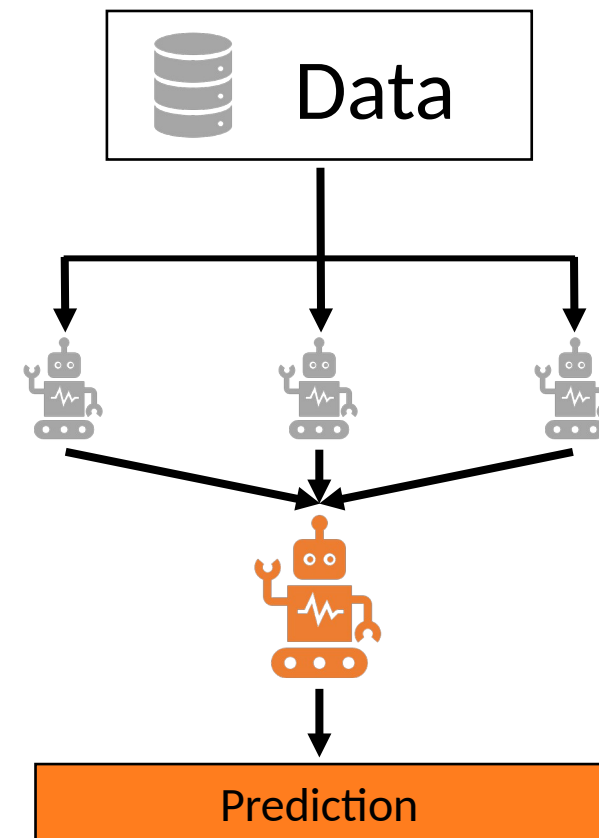
Begging



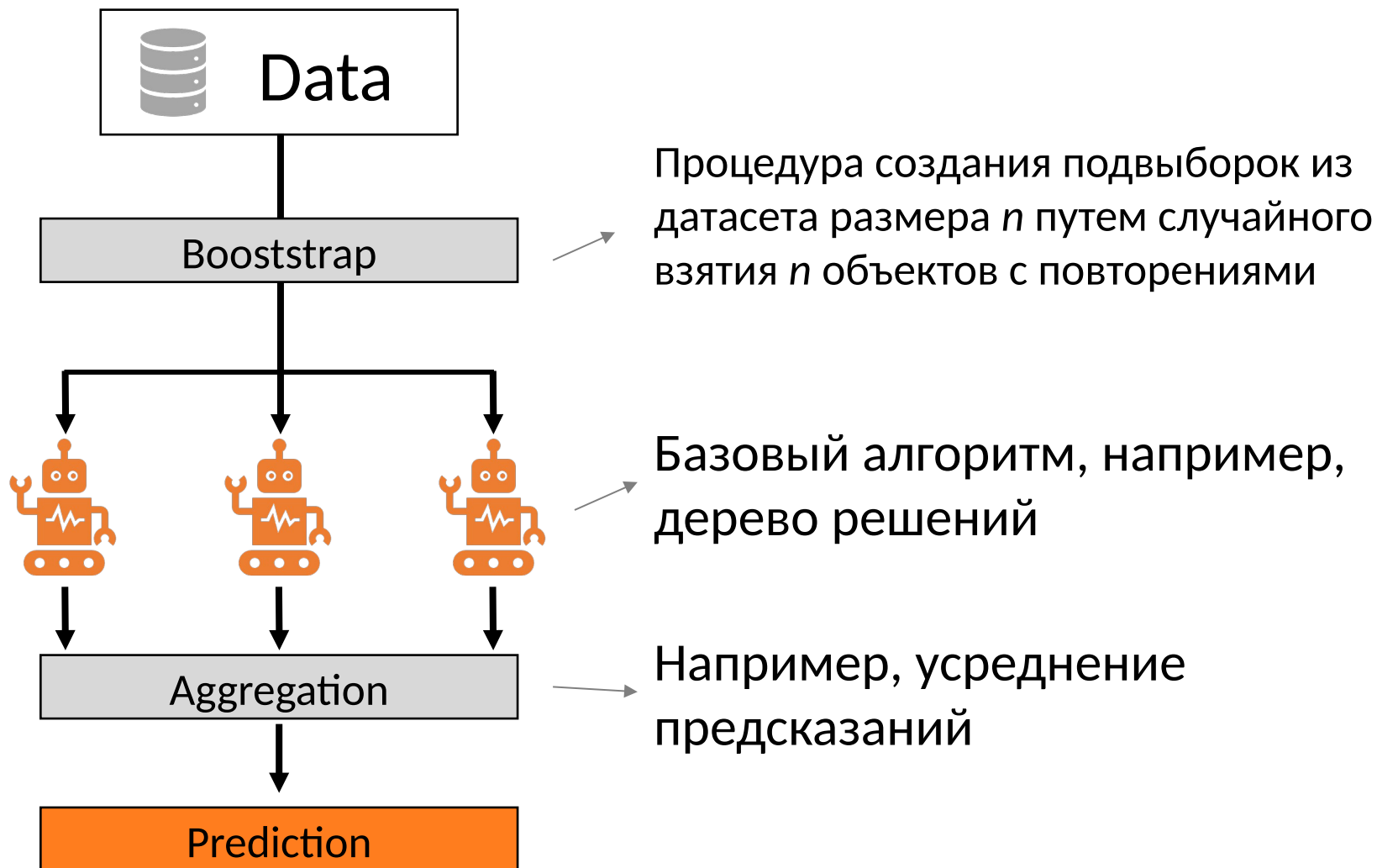
Boosting



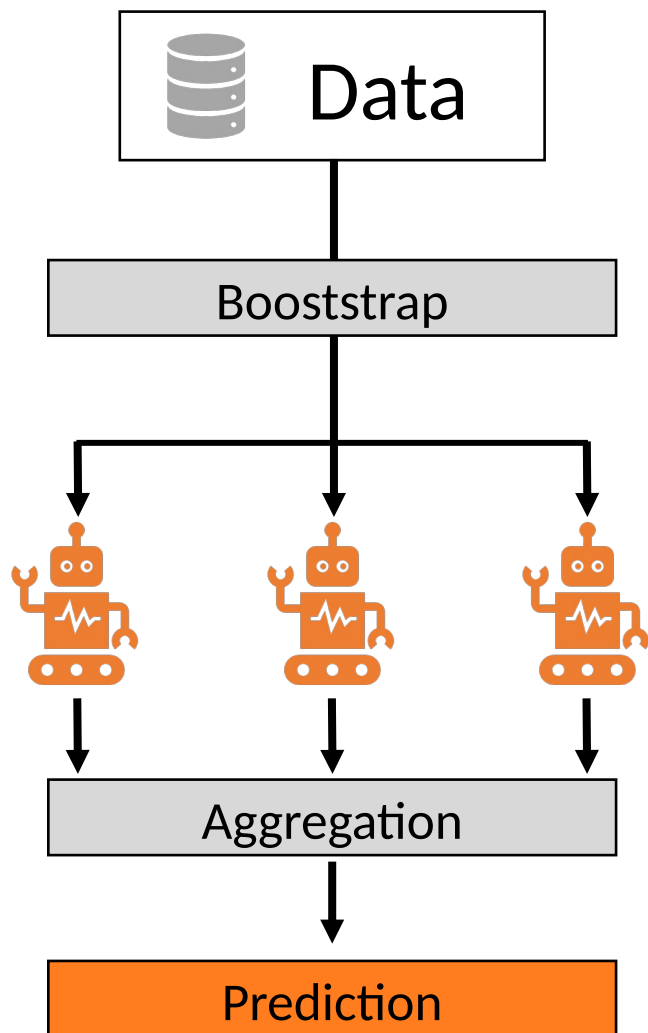
Stacking



Begging



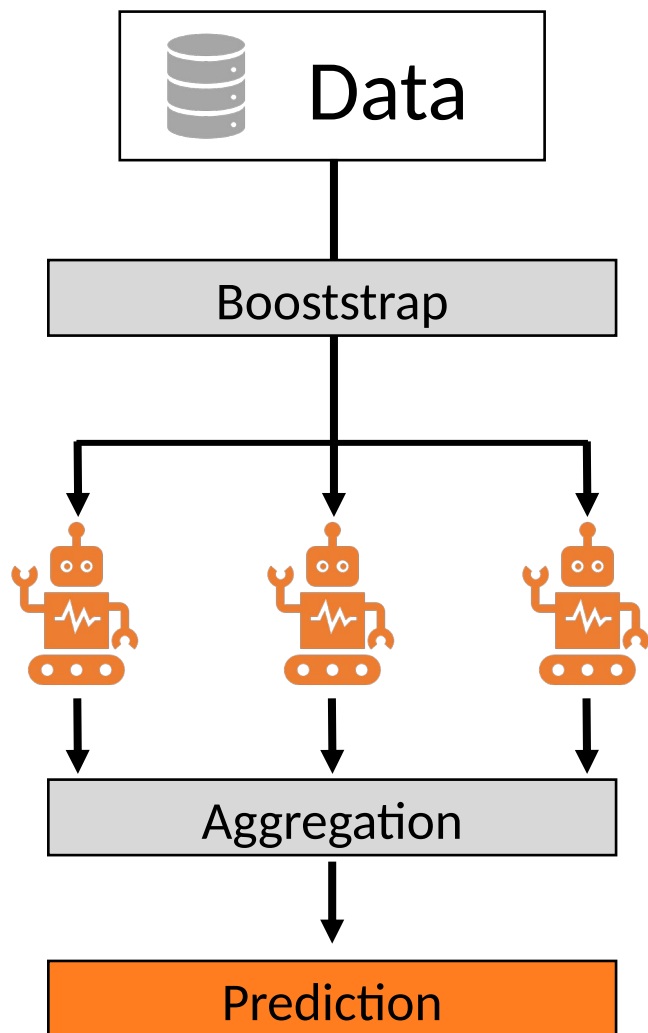
Begging



Преимущества:

- Почти не переобучается
- Базовые алгоритмы могут работать параллельно
- Довольно неплохо справляется с предсказаниями

Begging



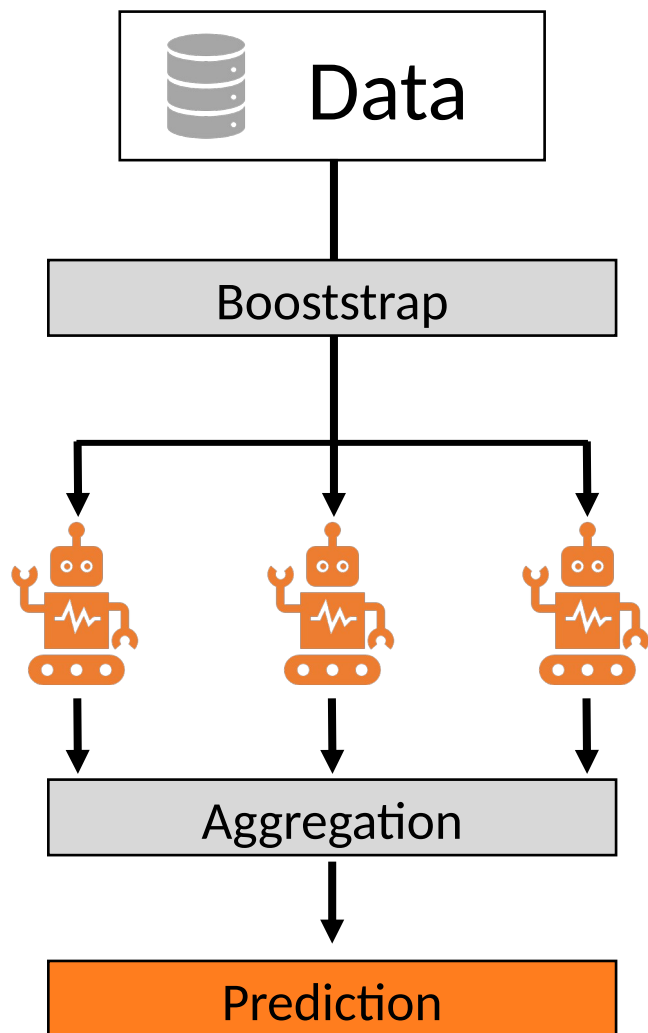
Преимущества:

- Почти не переобучается
- Базовые алгоритмы могут работать параллельно
- Довольно неплохо справляется с предсказаниями

Недостатки:

- Базовые алгоритмы работают независимо :(

Begging



Преимущества:

- Почти не переобучается
- Базовые алгоритмы могут работать параллельно
- Довольно неплохо справляется с предсказаниями

Недостатки:

- Базовые алгоритмы работают независимо :(



Идея: давайте учить базовые алгоритмы последовательно

Ансамбли (повторение)

Идея бустинга. AdaBoost

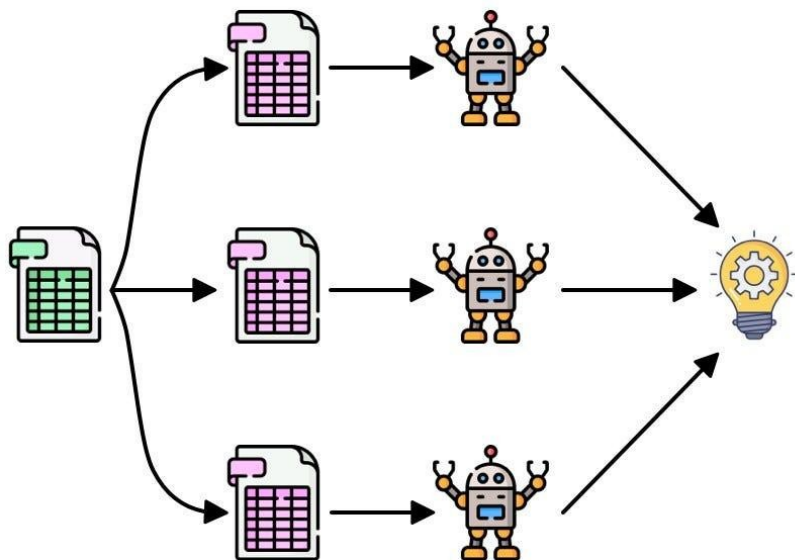
Градиентный бустинг

Семья бустингов

Стеккинг

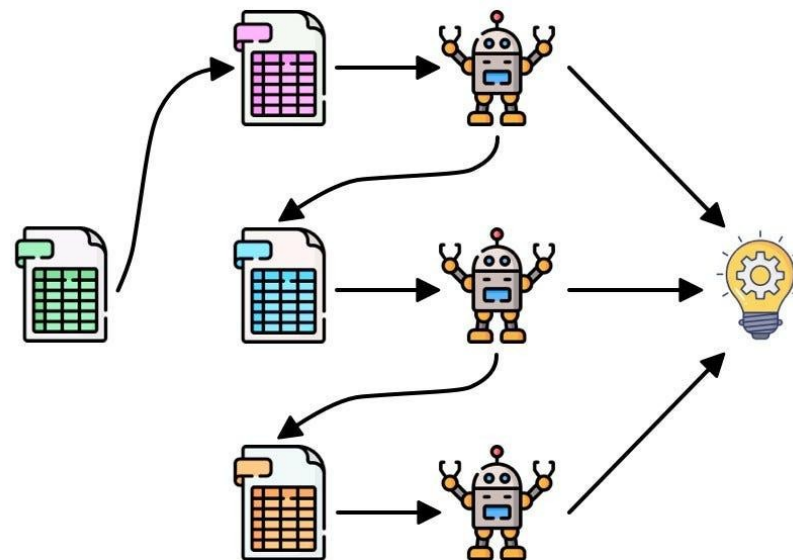
Begging vs Boosting

Begging



Параллельно

Boosting



Последовательно

Adaboost

AdaBoost(X, y, T, BaseAlgoClass): # T – количество базовых алгоритмов, $y \in \{-1, 1\}$

$W_t = [1/n] * n$ # W – вектор весов для объектов (инициализируем равномерно)

for t in range(T):

$base_t = \text{BaseAlgoClass}().\text{fit}(X, y, W_t)$ # minimize e_t

$e_t = W_t^T @ (base_t.\text{predict}(X) \neq y)$ # @ – скалярное произведение

$a_t = \frac{1}{2} \ln\left(\frac{1 - e_t}{e_t}\right)$ # весовой коэффициент для всего алгоритма

$W_t = W_t * \exp(-a_t * y * base_t.\text{predict}(X))$

$W_t.\text{normalize}()$ # нормализуем $W_t^i \in [0, 1]$, $\sum W_t^i = 1$

Adaboost.predict(X):

$y_{pred} = \text{sign}(\sum_{t=1}^T a_t * base_t.\text{predict}(X))$

return y_{pred}

Adaboost

AdaBoost(X

$W_t = [1/n]$

for t in range

$base$

$e_t =$

$a_t =$

$W_t =$

W_{t+1}

Adaboost.p

y_{pre}

retu

$\{-1, 1\}$

эрно)



Adaboost

AdaBoost(X, y, T, BaseAlgoClass): # T – количество базовых алгоритмов, $y \in \{-1, 1\}$

$W_t = [1/n] * n$ # W – вектор весов для объектов (инициализируем равномерно)

for t in range(T):

$h_t = \text{BaseAlgoClass}().\text{fit}(X, y, W_t)$ # minimize e_t

$e_t = W_t^T @ (h_t.\text{predict}(X) \neq y)$ # @ – скалярное произведение

$a_t = \frac{1}{2} \ln\left(\frac{1 - e_t}{e_t}\right)$ # весовой коэффициент для всего алгоритма

$W_t = W_t * \exp(-a_t * y * h_t.\text{predict}(X))$

$W_t.\text{normalize}()$ # нормализуем $W_t^i \in [0, 1], \sum W_t^i = 1$

Adaboost.predict(X):

$y_{pred} = \text{sign}(\sum_{t=1}^T a_t * \text{base}_t.\text{predict}(X))$

return y_{pred}

Ансамбли (повторение)

Идея бустинга. AdaBoost

Градиентный бустинг

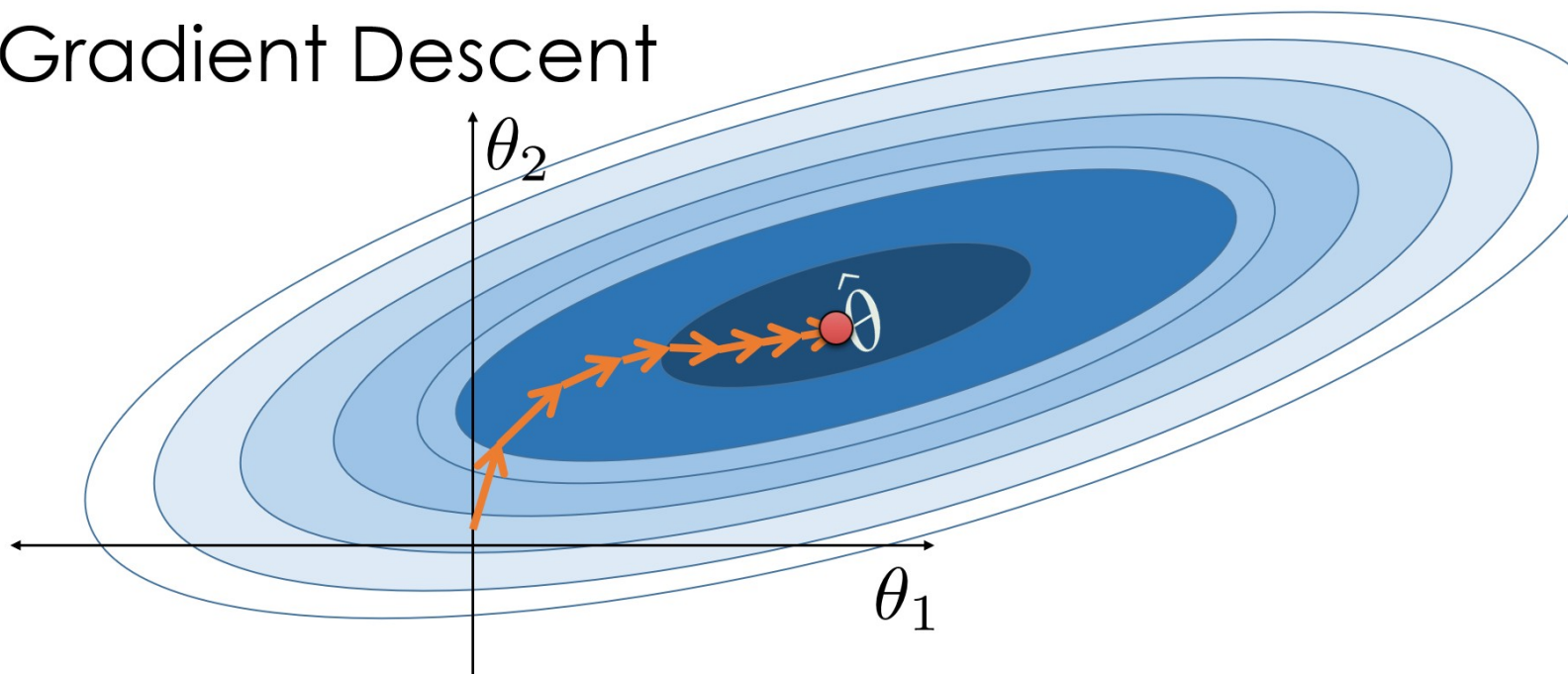
Семья бустингов

Стеккинг

Градиентный спуск

$$\theta_t = \theta_t - \gamma * \nabla_{\theta_t} Loss(X, y, \theta_t)$$

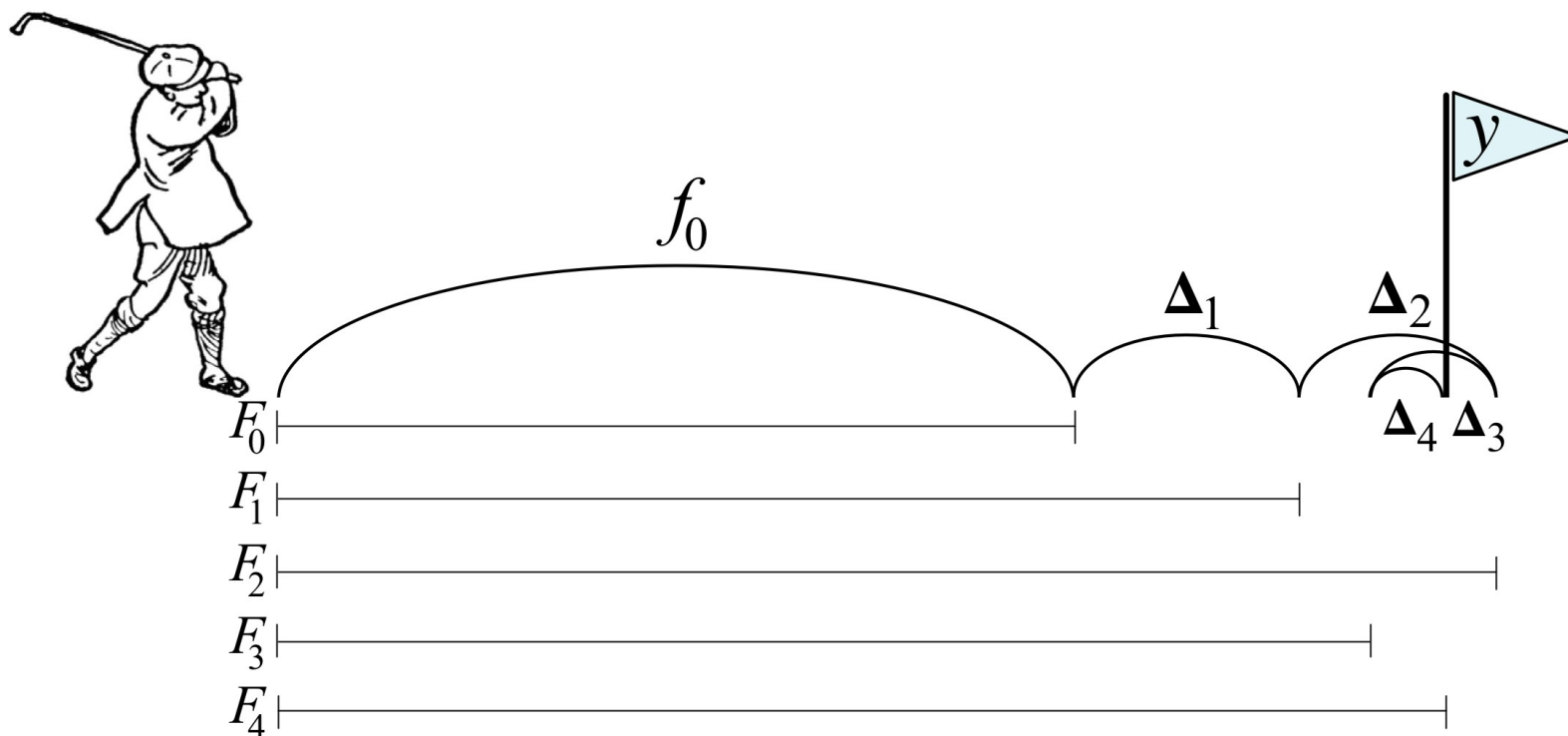
Gradient Descent



Градиентный спуск



Идея: Давайте обучать алгоритм на градиентах функции потерь



Градиентный спуск: алгоритм

$$\hat{f}(X) = \sum_{i=0}^{t-1} \hat{f}_i(X)$$

$$\rho_t, \theta_t = \arg \min_{\rho, \theta} E_{X,y} [L(y, \hat{f}(X) + \rho * h(X, \theta))]$$

$$\hat{f}_t(X) = \rho_t * h(X, \theta_t)$$

Градиентный спуск: алгоритм

$$\hat{f}(X) = \sum_{i=0}^{t-1} \hat{f}_i(X)$$

$$r_t = - \left[\frac{\partial L(y, f(X))}{\partial f(X)} \right]_{f(X)=\hat{f}(X)} \quad \# \text{ посчитаем псевдоостатки}$$

$$\theta_t = \arg \min_{\theta} (r_t - h(X, \theta))^2 \quad \# \text{ обучим базовую модель}$$

$$\rho_t = \arg \min_{\rho} L(y, \hat{f}(X) + \rho * h(X, \theta)) \quad \# \text{ подберем } \rho_t$$

$$\hat{f}_t(X) = \rho_t * h(X, \theta_t)$$

Градиентный спуск: алгоритм

GBM(X, y, T, BaseAlgoClass): # T – количество базовых алгоритмов, $y \in \{-1, 1\}$

$\hat{f}_0 = \operatorname{argmin}_{\gamma} L(y, \gamma)$ # инициализируем базовое предсказание константой

for t in range(T):

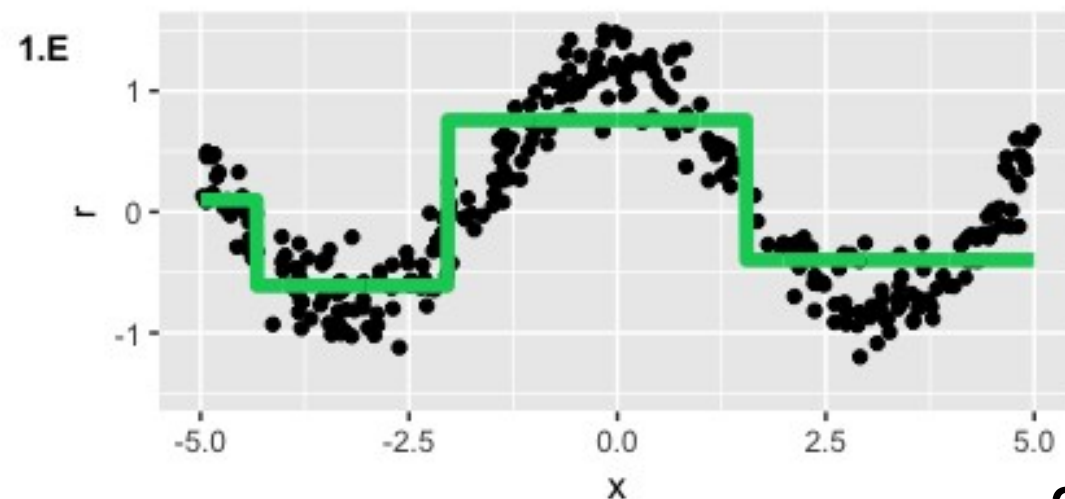
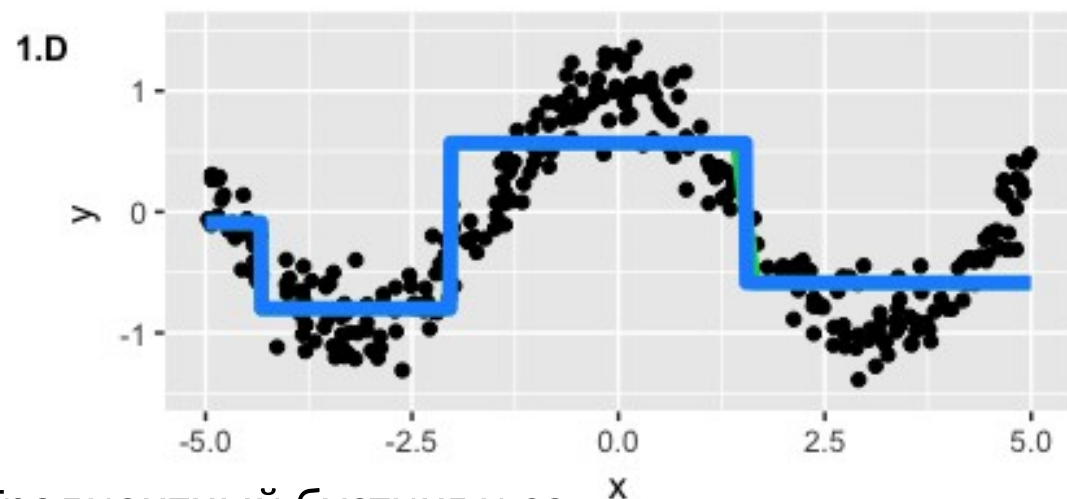
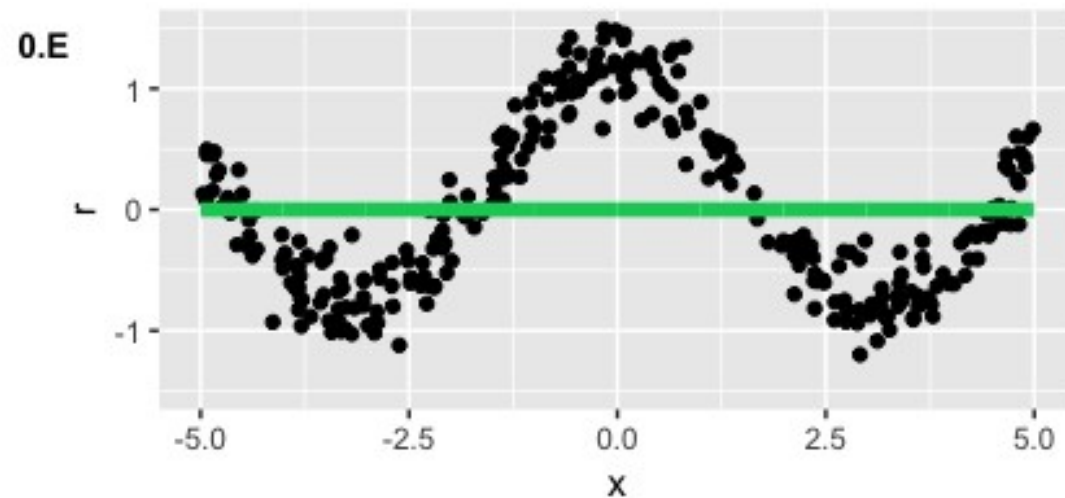
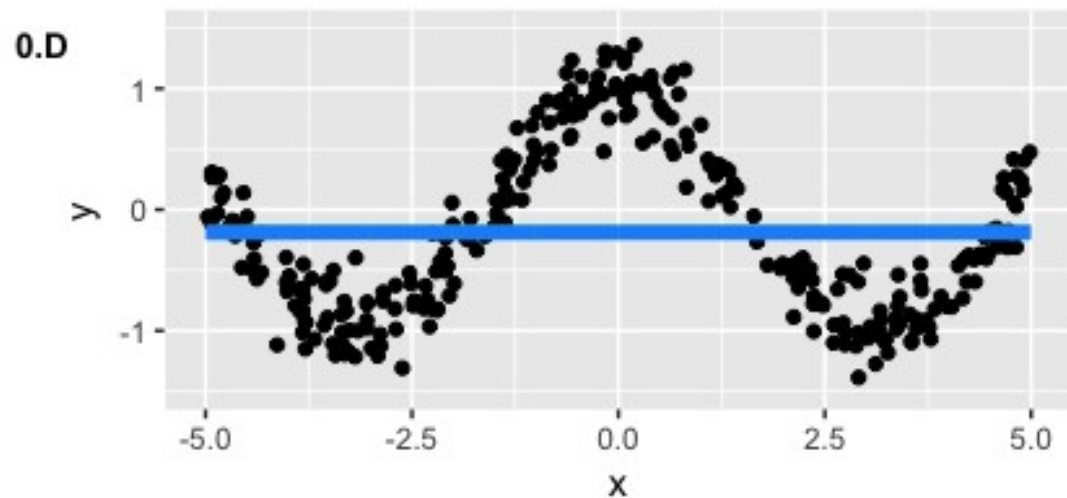
$r_t = - \frac{\partial L(y, f(X))}{\partial f(X)}$ # считаем псевдоостатки модели

$h_t = \text{BaseAlgoClass}().\text{fit}(X, r_t) = \operatorname{argmin}_{\theta} L(r_t, h_t(X, \theta))$ # обучаем базовую модель

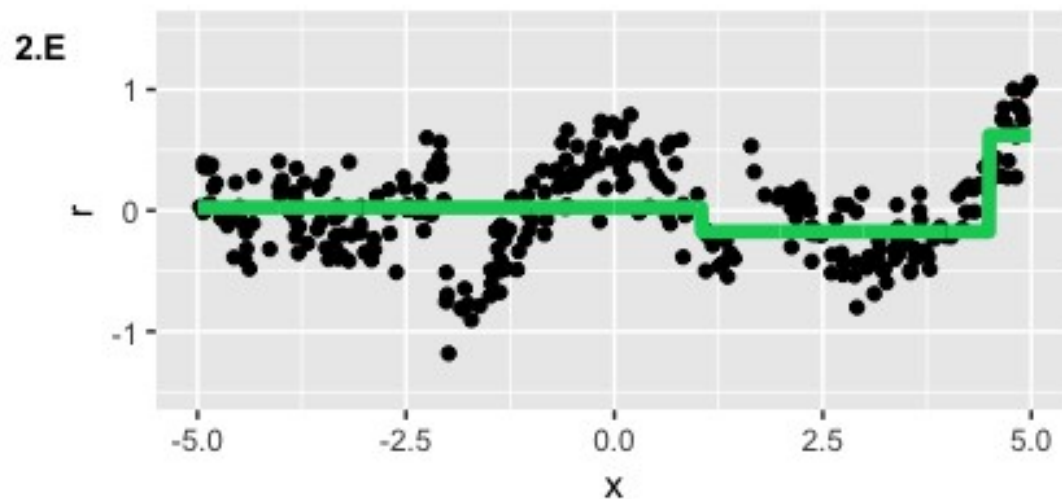
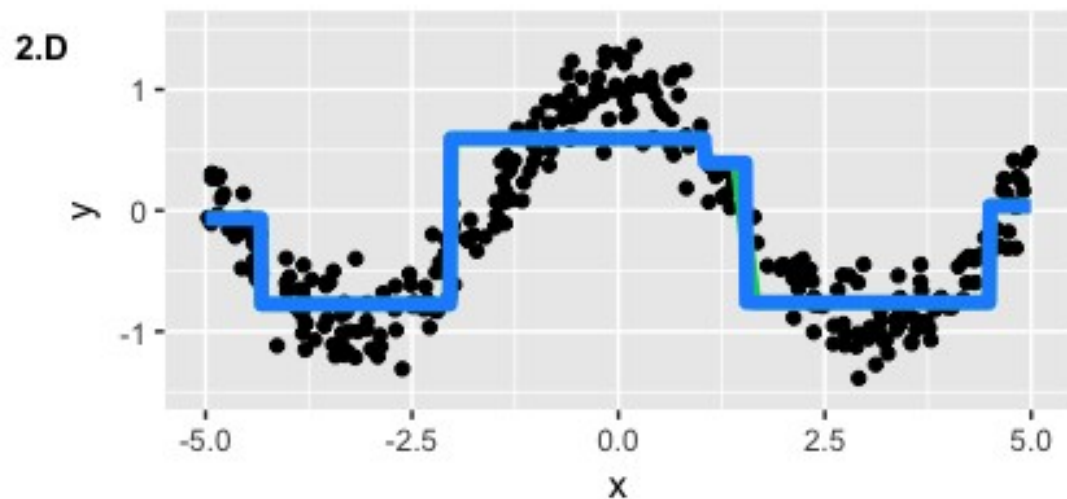
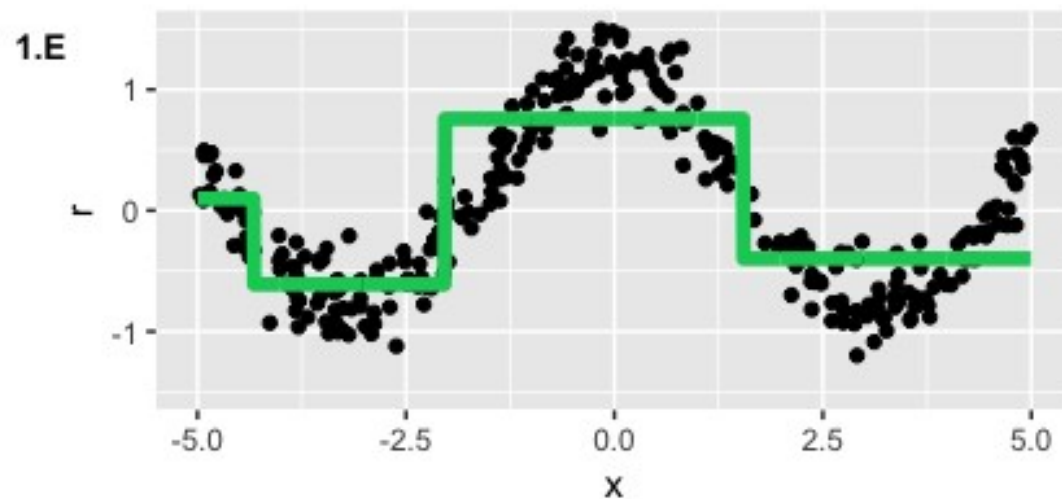
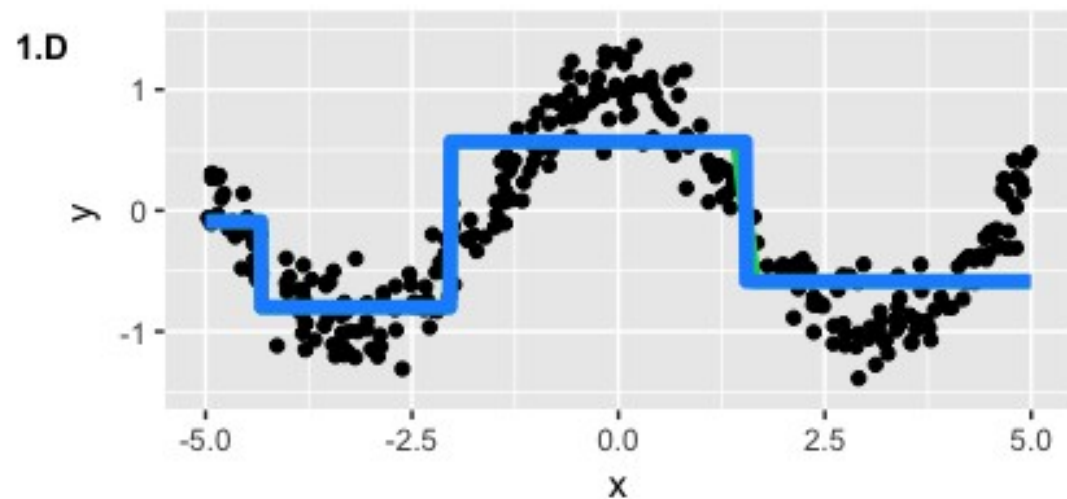
$\rho_t = \operatorname{argmin}_{\rho} L(y, \hat{f}(X) + \rho * h(X, \theta))$ # подбираем ρ_t - размер шага

$\hat{f}(X) = \hat{f}(X) + \rho_t * h_t(X)$ # обновляем модель

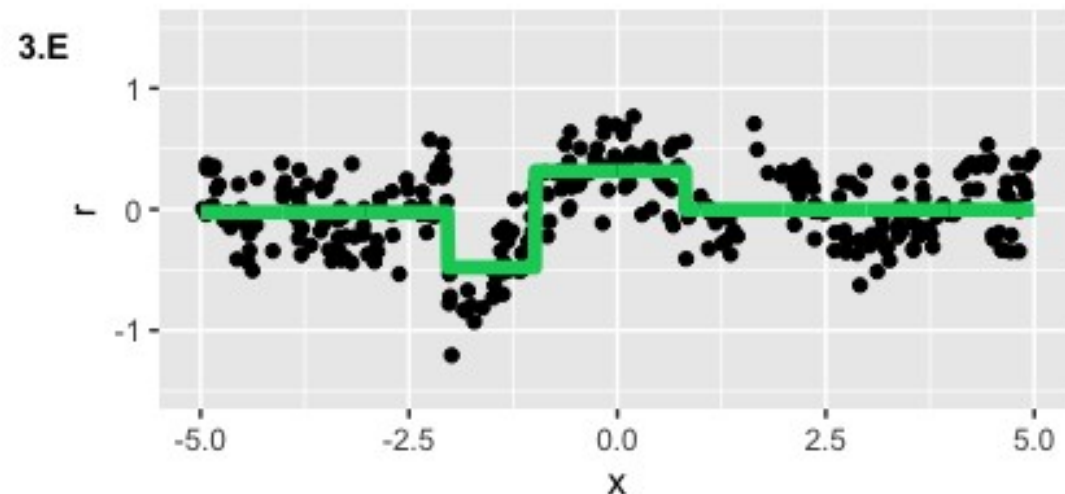
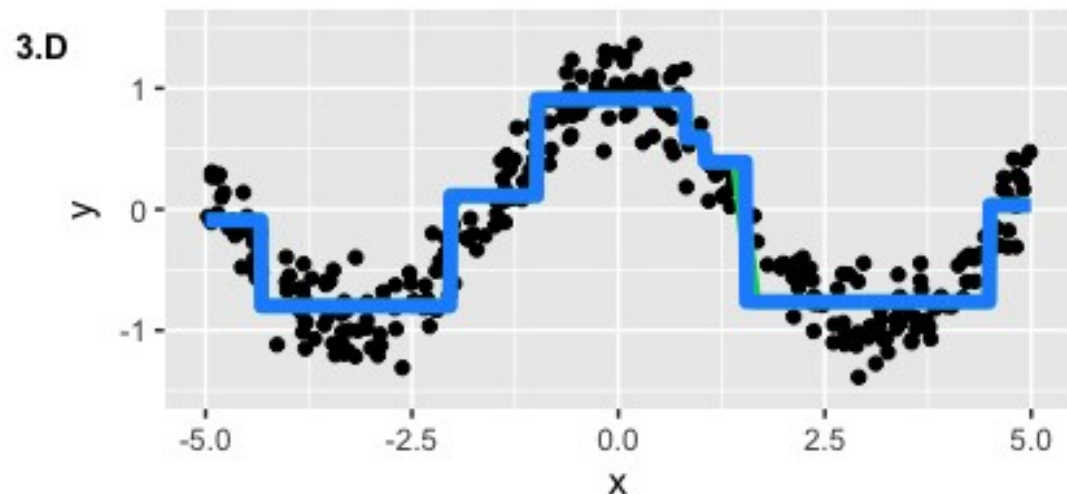
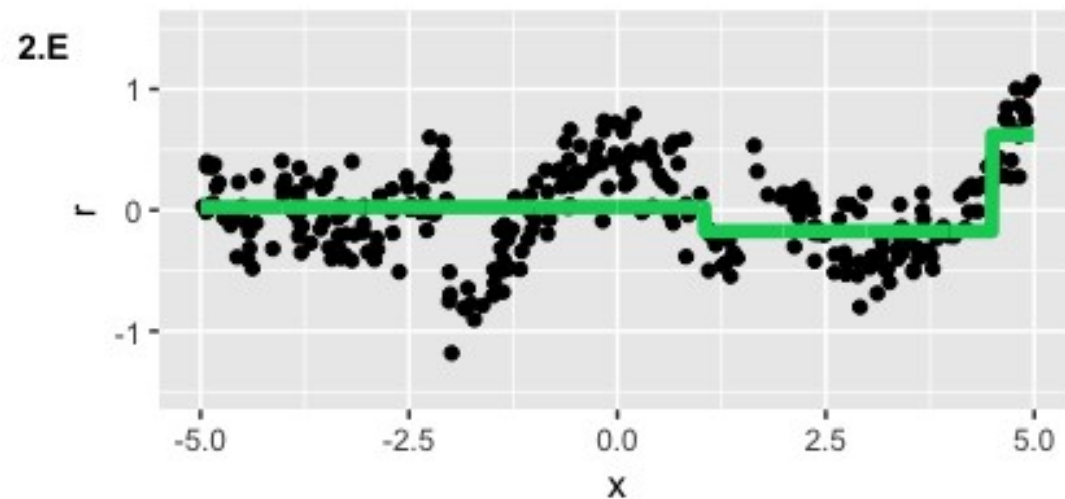
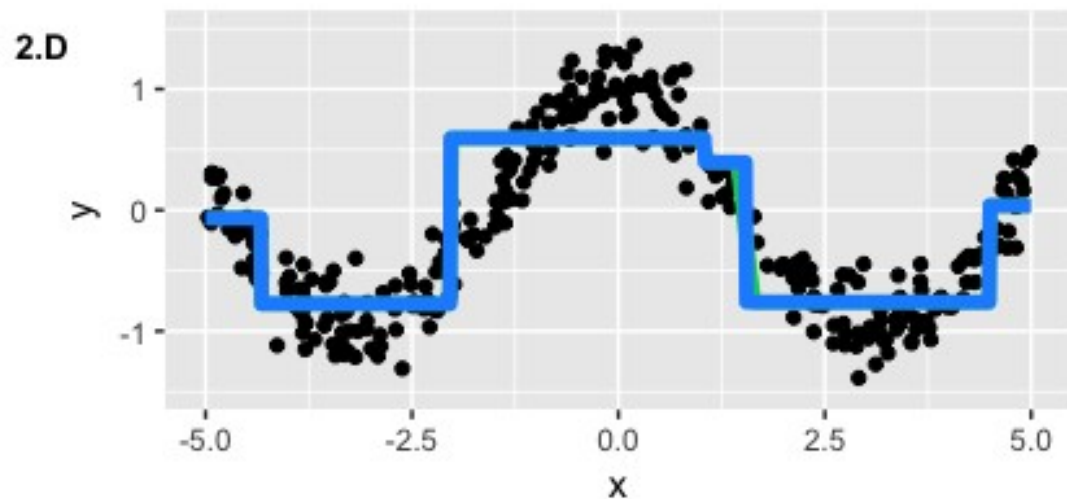
Градиентный спуск: визуализация



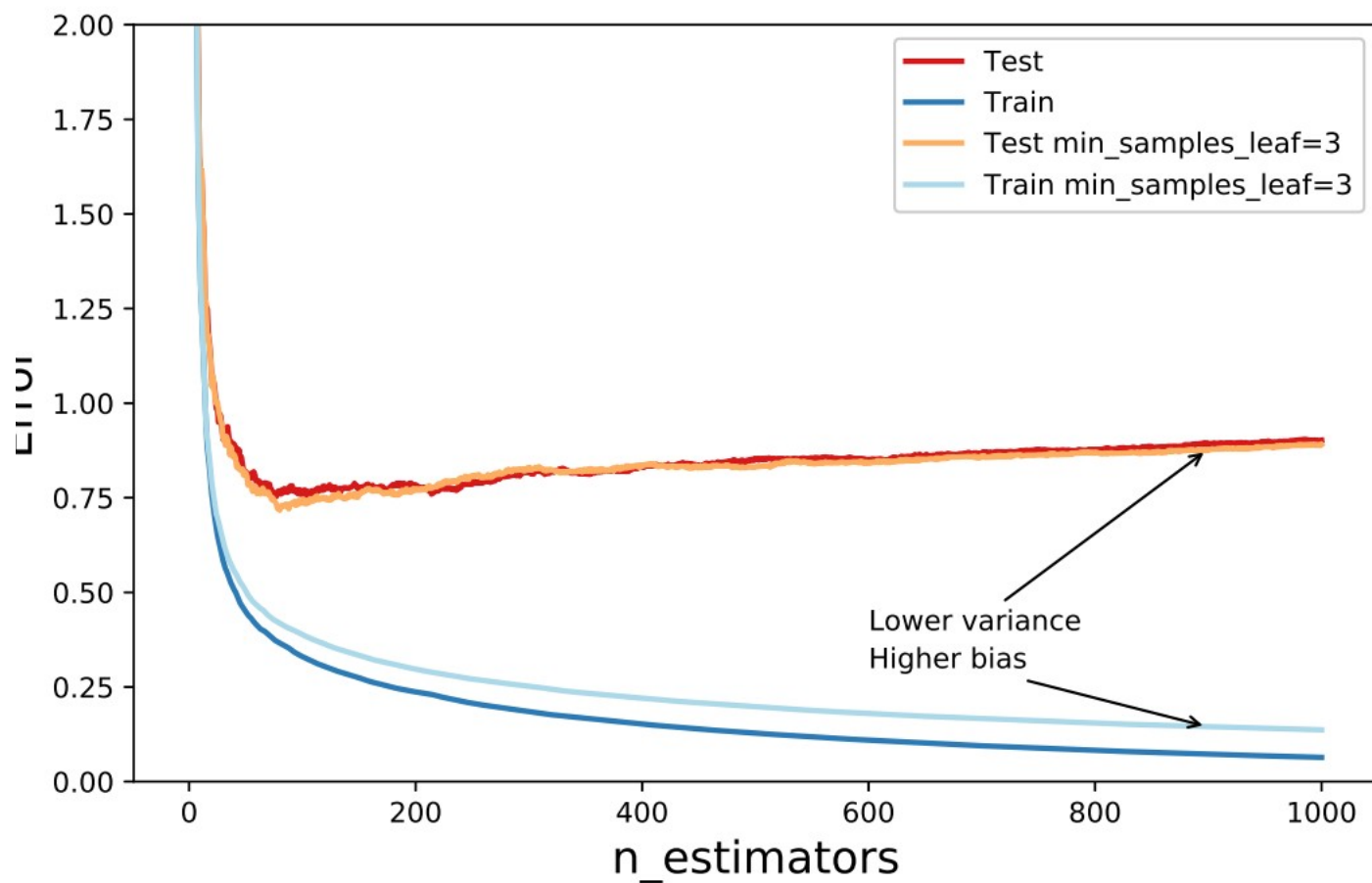
Градиентный спуск: визуализация



Градиентный спуск: визуализация



Градиентный спуск: нюансы



Преимущества:

- Низкое смещение
- SOTA для табличных данных

Недостатки:

- Нестабильное обучение
- Склонен к переобучению

Ансамбли (повторение)

Идея бустинга. AdaBoost

Градиентный бустинг

Семья бустингов

Стеккинг

Главные модификации бустинга

XGBoost



CatBoost



LightGBM

Главные модификации бустинга



XGBoost

XGBoost



Идея: использование сбалансированных деревьев

$$w_j = \text{learning rate} \times \frac{\sum_{i \in I_j} \frac{\partial \text{loss}}{\partial (\hat{y}=0)}}{\sum_{i \in I_j} \left(\frac{\partial^2 \text{loss}}{\partial (\hat{y}=0)^2} \right) + \lambda}$$

$$\text{total_loss} = \sum_{i=1}^N \text{loss}(y_i, F(x_i)) + \gamma \sum_{k=1}^K T_k + \frac{1}{2} \lambda \sum_{k=1}^K \|w_k\|^2$$

T_k - количество листьев в k -ом дереве

w_k - выходы всех листьев k -ого дерева решений
Градиентный бустинг и со.

LightGBM



Идея: Вместо градиентного спуска можно использовать **стохастический** градиентный спуск

$$\theta_t = \theta_t - \gamma * \nabla_{\theta_t} Loss(x_i, y_i, \theta_t)$$

CatBoost



CatBoost

1. Строит деревья по принципу: «Все вершины одного уровня имеют одинаковый предикат»
2. Упорядоченное target-кодирование на категориальных признаках высокой размерности
3. Бустинг с упорядочиванием обучающих примеров
4. CatBoost позволяет проводить обучение на нескольких GPU.
5. Умеет под капотом обрабатывать пропущенные значения.

Ансамбли (повторение)

Идея бустинга. AdaBoost

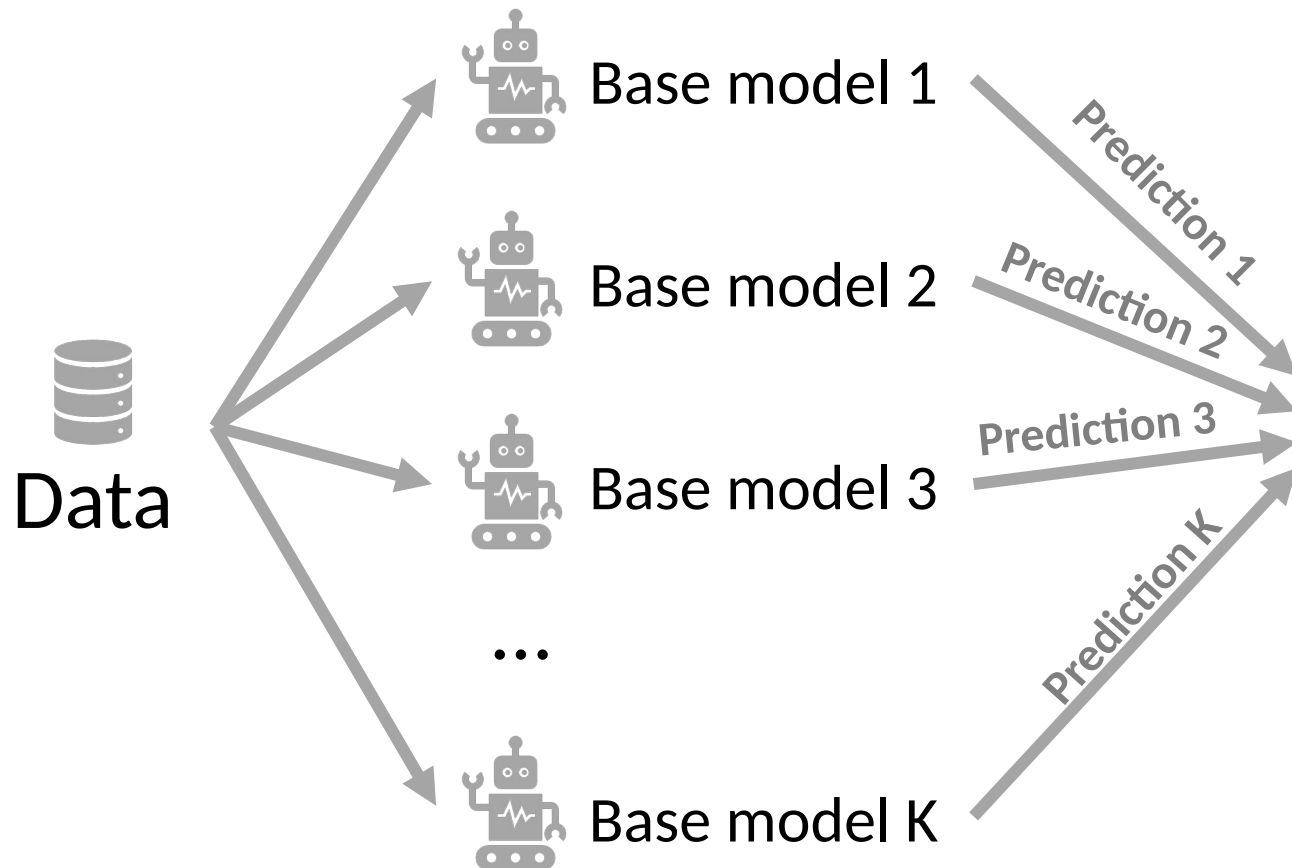
Градиентный бустинг

Семья бустингов

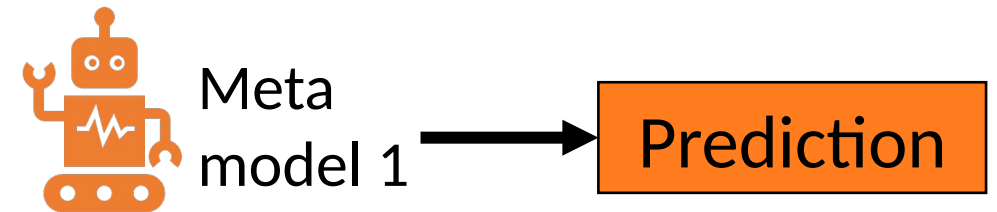
Стеккинг

Стеккинг

Тренировка и предсказание
базовых моделей



Обучение метамоделей на
предсказания базовых



Стеккинг



Выводы

1. Бустинг это **ансамблевая модель**, где каждая **базовая модель улучшает** работу **предыдущих**
2. Бустинг работает с меньшим смещением по сравнению с лесом, но способен переобучаться
3. Существует много модификаций бустинга, например, XGBoost, CatBoost, LightGBM
4. Стекинг это ансамбль, где метамодель учится на предсказаниях базовых моделей